ADR-002 — Apache Kafka como backbone de eventos

Contexto

Um mesmo pedido precisa chegar em segundos a vários consumidores independentes: a camada bronze (reprocessamento), o detector de fraude, o serviço de estoque, o painel ao vivo e o gatilho de carrinho abandonado. Cada um lê no seu ritmo, e qualquer um pode ficar fora do ar por um tempo, por exemplo durante uma atualização, sem perder pedidos.

Na Black Friday chegam cerca de 300 mil pedidos por dia, e cada pedido gera mais de um evento (o pedido, seus itens e as mudanças de status). Precisamos de um broker que guarde os eventos por um tempo, permita reler desde o início para reprocessar e cresça junto com o volume.

Decisão

Usamos o Apache Kafka, em modo KRaft (sem ZooKeeper), como backbone de eventos da plataforma.

Há um tópico por tabela capturada (`shopvibe.shopvibe.orders` e `shopvibe.shopvibe.order_items`), com 3 partições cada no laboratório. A chave de cada mensagem é a chave primária da linha, padrão do Debezium, então todos os eventos de uma mesma linha caem na mesma partição e mantêm a ordem. Cada consumidor usa o seu próprio consumer group e lê os mesmos eventos sem atrapalhar os demais. A retenção padrão do Kafka (7 dias) permite reler os eventos recentes, e a camada bronze guarda o histórico longo.

Alternativas consideradas

O Kafka foi o único a combinar releitura de eventos, vários consumidores independentes, integração nativa com o Debezium e licença open source.

Alternativa	Por que não foi escolhida
RabbitMQ	Em filas clássicas a mensagem some após o consumo; reler exige o recurso mais recente de streams. O Debezium roda nativamente sobre o Kafka Connect
Redpanda	Compatível com a API do Kafka, mas o broker usa licença source-available (BSL), fora da restrição de open source
Apache Pulsar	Open source e capaz, mas tem mais componentes para operar (BookKeeper) e custo de aprendizado alto para o prazo do projeto
Gravar direto na bronze, sem broker	Elimina os consumidores em tempo real (fraude, estoque, painel) e volta a depender de leitura em

Alternativa	Por que não foi escolhida
	lote
Serviço de streaming de nuvem (ex.: Kinesis)	Proprietário e cobrado por uso, fora da restrição de open source

Consequências

O Kafka desacopla produtor e consumidores e permite reprocessar, mas é o componente mais pesado de operar da solução e, no laboratório, não oferece durabilidade real.

Positivas

- Cada novo consumidor (fraude, estoque, painel) entra sem alterar o produtor nem os demais.
- Reprocessamento por reposicionamento de offsets, sem consultar o PostgreSQL de novo.
- Escala por partições e por instâncias de consumidor dentro de um mesmo grupo.
- Ecossistema pronto: Kafka Connect, Kafka UI, e processadores de stream como Flink e Spark.

Negativas

- É preciso operar o cluster: disco, retenção, monitoramento do atraso dos consumidores e atualizações.
- A ordem só é garantida dentro de uma partição; eventos de `orders` e de `order_items` não têm ordem entre si, e a junção fica para o processamento posterior.
- O laboratório usa um único broker com fator de replicação 1, o que não protege contra a perda do broker (ver ADR-004).
- Os eventos estão em JSON sem schema, para facilitar a leitura no vídeo; em produção o ideal é Avro com Schema Registry.
- A retenção de 7 dias não substitui a camada bronze como fonte de reprocessamento de longo prazo.